

Cryochamber

A hibernation chamber for AI agents

Product Usage Guide

For operators, developers, and curious readers

Contents

1. What Is Cryochamber?	3
2. Core Concepts	3
2.1. Session	3
2.2. Daemon	3
2.3. IPC	3
2.4. TODO List	3
2.5. Inbox / Outbox	3
3. Architecture	4
4. Session Lifecycle	5
5. Getting Started	5
5.1. Install	5
5.2. Create a Project	5
5.3. Start the Daemon	6
5.4. Monitor Progress	6
5.5. Send & Receive Messages	6
5.6. Stop	6
6. The Agent Protocol	6
7. Human ↔ Agent Communication	7
7.1. Direct Messaging (local)	7
7.2. GitHub Discussions Sync	7
7.3. Zulip Sync	8
8. Configuration Reference	8
9. Runtime File Reference	8
10. A Complete Example Walk-Through	9
11. Provider Rotation	10
12. Tips for Developers	10

1. What Is Cryochamber?

Cryochamber is an open-source AI-agent scheduler written in Rust. It solves a fundamental problem: *large tasks take longer than a single LLM context window*.

Modern AI agents (Claude, OpenCode, GPT-4 ...) can run code, browse the web, and write software — but they time out, hit rate limits, or simply need to pause and think overnight. Cryochamber gives them a persistent harness:

⚠ The Problem

- Agent times out mid-task
- No memory between runs
- Human must restart manually
- API rate limits block overnight work

✓ What Cryochamber Provides

- Scheduled wake-ups via TODO list
- Persistent logs & notes across sessions
- Human↔agent messaging
- Automatic retry on crash

The name reflects the metaphor: the agent is placed in a **cryochamber** between sessions — state is frozen, resources are freed, and it wakes up exactly when needed.

2. Core Concepts

2.1. Session

A **session** is one run of the agent process. Every session has a number, a task description, a start timestamp, and an end outcome (`complete`, `exit <N>`, `timeout`, ...). Sessions are delimited in `cryo.log` by `--- CRYO SESSION N --- / --- CRYO END ---` markers.

2.2. Daemon

The **daemon** is a long-running background process managed by the OS service layer (launched on macOS, systemd on Linux, SCM on Windows). It:

- sleeps until the earliest pending TODO time;
- spawns the agent subprocess for each session;
- enforces a configurable session timeout;
- retries crashed agents with exponential backoff (5 s → 15 s → 60 s);
- watches `messages/inbox/` for reactive wakes.

2.3. IPC

The agent subprocess communicates with the daemon over a platform IPC channel (Unix domain socket on Unix, named pipe on Windows) using the `cryo-agent` CLI. This keeps the agent decoupled: it just runs shell commands.

2.4. TODO List

The agent controls its own wake schedule through `todo.json`. Adding a TODO with a future `--at` time causes the daemon to sleep until that time and then re-spawn the agent. If no TODO is pending the daemon idles.

2.5. Inbox / Outbox

A simple file-based message bus lives in `messages/`. The human writes to `inbox/` (via `cryo send`); the agent reads it at session start. The agent writes replies to `outbox/`; the human reads them (via `cryo receive`). Channels such as GitHub Discussions and Zulip mirror this bus over the internet.

3. Architecture

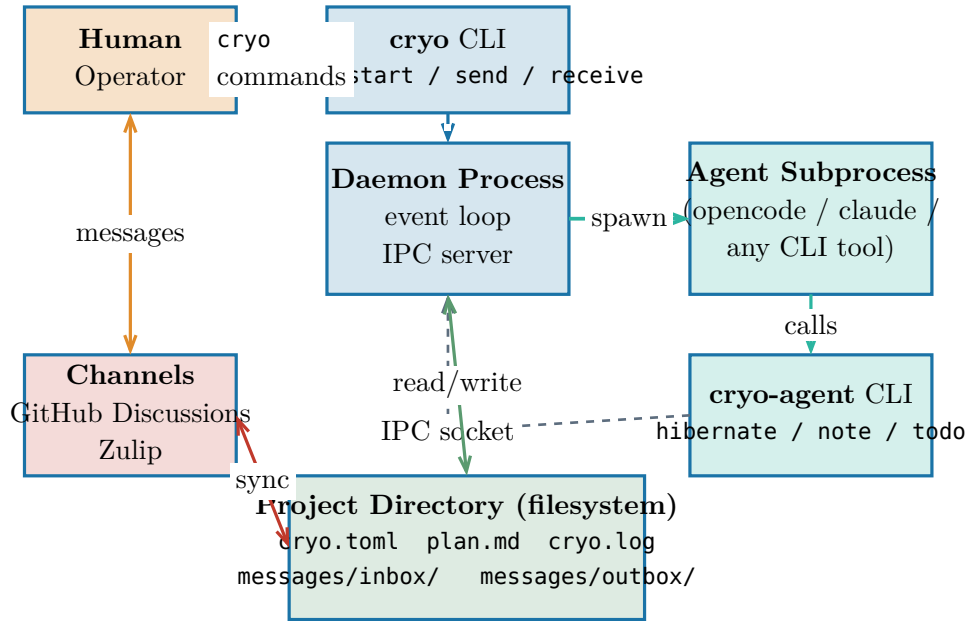


Figure 1: Component architecture of Cryochamber.

The diagram shows the five main actors:

Actor	Role
cryo CLI	Operator's entry point — initialise projects, start/stop the daemon, view logs, send messages.
Daemon	Long-running background process. Manages the session lifecycle, IPC server, and wake logic.
Agent subprocess	The actual AI tool (e.g. <code>opencode run</code>). Completely swappable — any CLI program works.
cryo-agent CLI	Agent's toolbox. Sends commands over IPC: hibernate, add notes, schedule todos, send messages.
Channels	Optional bridges (GitHub Discussions, Zulip) that mirror the inbox/outbox to internet services.

4. Session Lifecycle

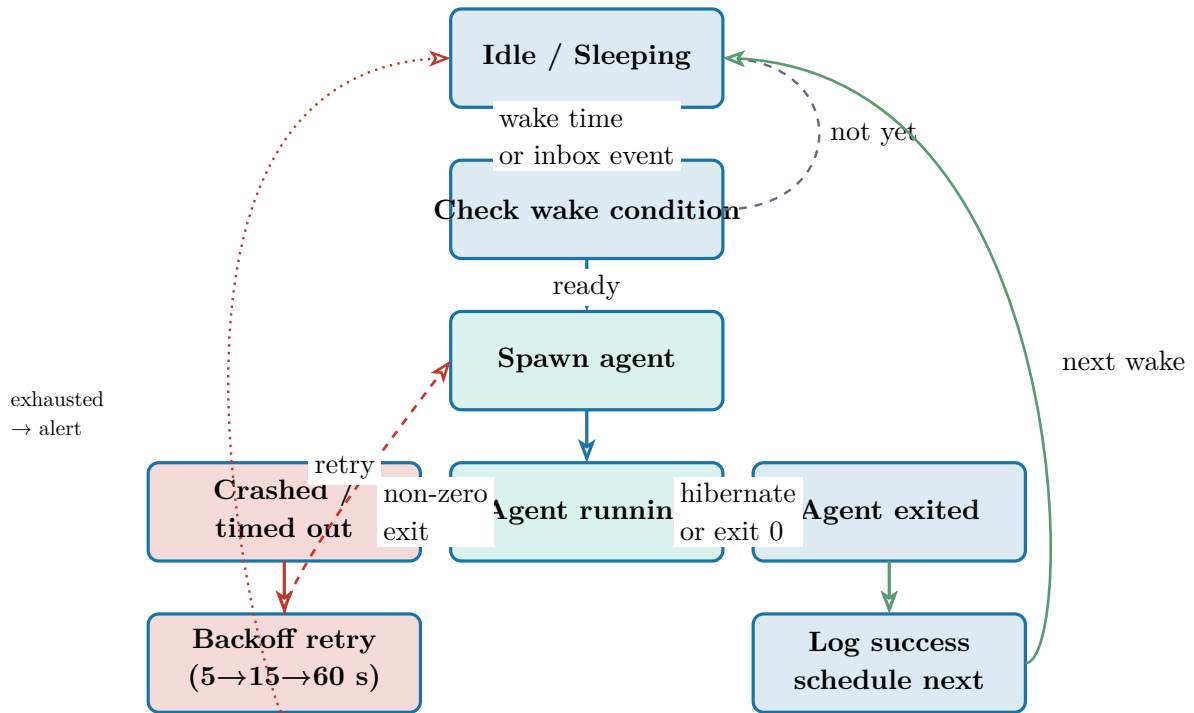


Figure 2: Daemon session lifecycle state machine.

The daemon loops indefinitely. Key points:

- **Normal exit:** the agent calls `cryo-agent hibernate`; the daemon logs the session and schedules the next wake from `todo.json`.
- **Crash/timeout:** the daemon retries with exponential backoff. After `max_retries` consecutive failures it fires a **fallback alert** (dead-man switch) to the outbox.
- **Reactive wake:** a new file in `messages/inbox/` or a `cryo wake` command can cut the sleep short.

5. Getting Started

5.1. Install

```
cargo install cryochamber          # from crates.io
# or, from source:
git clone https://github.com/GigggleLiu/cryochamber
cd cryochamber && cargo install --path .
```

This installs four binaries: `cryo`, `cryo-agent`, `cryo-gh`, `cryo-zulip`.

5.2. Create a Project

```
mkdir my-ai-project && cd my-ai-project
echo "# Plan\nWrite the plan here." > plan.md
cryo init --agent "opencode run"
```

`cryo init` creates:

- `cryo.toml` — project configuration
- `plan.md` — (already present) read by the agent at session start
- `CLAUDE.md` / `AGENTS.md` — cryochamber protocol injected into the agent's context

5.3. Start the Daemon

```
cryo start                # uses agent from cryo.toml
cryo start --agent "claude -p"  # override agent for this run
```

On macOS/Linux this installs a **launchd** / **systemd** service that survives reboots. Set `CRYO_NO_SERVICE=1` to skip service installation and run as a plain background process instead.

5.4. Monitor Progress

```
cryo status      # daemon state, session number, next wake time
cryo watch       # live-tail of cryo.log (tail -f style)
cryo watch --all  # from the beginning of the log
cryo log         # dump the full structured log
```

5.5. Send & Receive Messages

```
cryo send "Please review the PR before the next session"
cryo receive                # read agent replies from outbox
```

`cryo send` places a Markdown message in `messages/inbox/`. The daemon wakes the agent (if sleeping) and the agent reads the message at the top of its next session.

5.6. Stop

```
cryo cancel      # stop daemon + remove service
cryo clean --force  # cancel + delete all runtime files
```

6. The Agent Protocol

When the daemon spawns the agent it injects a **cryochamber protocol block** at the top of the prompt. This block instructs the agent to use `cryo-agent` subcommands for structured communication.

 **Session Workflow (as seen by the agent)**

→ **Step 1 — Orient:** Read `plan.md`, run `cryo-agent todo list`, check inbox.

→ **Step 2 — Work:** Do the task. Reply to inbox messages with `cryo-agent reply`.

→ **Step 3 — Record:** `cryo-agent note "what I did and what's next"`.

→ **Step 4 — Schedule:** `cryo-agent todo add "next task" --at <TIME>` using `cryo-agent time "+2 hours"`.

→ **Step 5 — Hibernate:** `cryo-agent hibernate --summary "..."` (must be the last command).

cryo-agent command	What it does
hibernate --summary "..."	End the session. Daemon logs the outcome and schedules the next wake.
hibernate --complete	Mark the overall project as finished.
hibernate --exit 1 --summary "..."	Signal a failure / blocked state.
note "text"	Append a note to the session log. Readable by the next session via <code>parse_latest_session_notes</code> .
send "message"	Write a message to <code>messages/outbox/</code> .

reply "message"	Reply to the current inbox messages.
receive	Read messages/inbox/ (local — no daemon needed).
alert <action> <target> "msg"	Set a dead-man switch: if the agent doesn't wake on time, fire <action> to <target>.
todo add "task" --at <ISO>	Add a scheduled TODO item. The daemon wakes at the earliest pending time.
todo list	List all pending TODO items.
todo done <id>	Mark a TODO item as completed.
time	Print current ISO8601 local time.
time "+2 hours"	Compute and print a future ISO8601 time.

7. Human ↔ Agent Communication

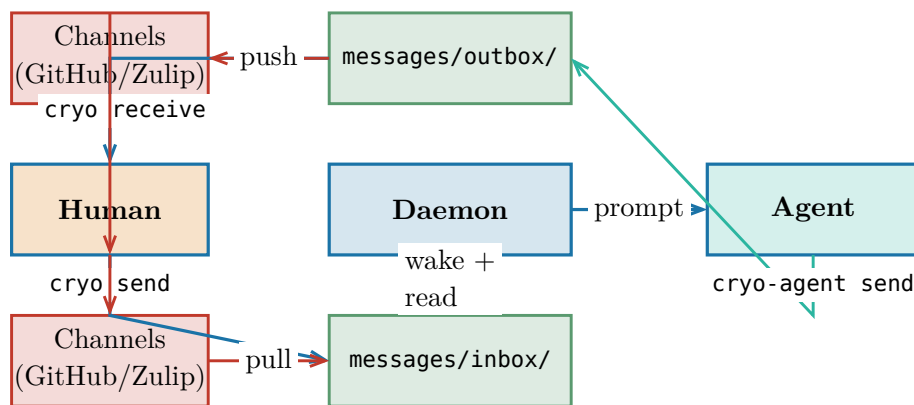


Figure 3: Human ↔ agent communication flow.

7.1. Direct Messaging (local)

The simplest channel — no external accounts needed:

```
# Human writes
cryo send "Please add unit tests for the auth module"

# Human reads replies
cryo receive
```

7.2. GitHub Discussions Sync

```
cryo-gh init --repo owner/repo --discussion 42
cryo-gh sync      # start background sync daemon
cryo-gh status    # check sync configuration
cryo-gh unsync    # stop background sync
```

New comments on the GitHub Discussion are pulled to `messages/inbox/` on each sync cycle. Session summaries are pushed back as new comments.

7.3. Zulip Sync

```
cryo-zulip init --config ~/.zuliprc --stream "dev"
cryo-zulip sync      # start background sync daemon
cryo-zulip status    # check sync state
```

Messages posted in the configured Zulip stream appear in the inbox; the agent's replies are posted back to the stream.

8. Configuration Reference

The project configuration lives in `cryo.toml`:

```
agent = "opencode run"           # default agent command
max_retries = 3                  # crash retries before fallback alert
max_session_duration = 1800      # session timeout in seconds (0 = unlimited)
watch_inbox = true               # inotify-based reactive wake on new inbox msg
fallback_alert = "outbox"        # "outbox" | "notify" | "none"
report_interval_hours = 24       # 0 = disable periodic reports
report_time = "09:00"           # wall-clock time for daily report

# Multiple providers for automatic rotation on failure:
[[providers]]
name = "primary"
env = { OPENAI_API_KEY = "sk-..." }

[[providers]]
name = "fallback"
env = { ANTHROPIC_API_KEY = "sk-ant-..." }
```

Key	Default	Description
agent	opencode run	Shell command used to spawn the agent subprocess.
max_retries	3	Consecutive crash retries before giving up and sending a fallback alert.
max_session_duration	0	Maximum seconds per session. 0 means unlimited.
watch_inbox	true	Use filesystem events to wake the daemon immediately on new inbox message.
fallback_alert	"outbox"	What to do when retries are exhausted: write to outbox, send desktop notification, or nothing.
report_interval_hours	0	Period for periodic session-count desktop notifications. 0 disables.
report_time	"09:00"	Wall-clock time to align daily reports to.
providers	[]	List of AI provider configurations rotated on failure. Each entry can set env vars.

9. Runtime File Reference

File / Directory	Purpose
cryo.toml	Project config. Hand-edited. Created by <code>cryo init</code> .

plan.md	Task description injected into every agent session.
CLAUDE.md / AGENTS.md	Cryochamber protocol for the agent (auto-generated by <code>cryo init</code>).
timer.json	Runtime state: session number, PID lock, retry count, CLI overrides. Written by daemon.
cryo.log	Append-only structured event log. Read by <code>cryo log</code> / <code>cryo watch</code> .
cryo-agent.log	Raw stdout/stderr from the agent subprocess.
todo.json	Pending TODO items. Drives wake schedule.
messages/inbox/	Incoming messages for the agent. Written by <code>cryo send</code> and channel sync.
messages/outbox/	Agent replies. Read by <code>cryo receive</code> and pushed by channel sync.
messages/inbox/archive/	Processed (already read) inbox messages.
.cryo/cryo.sock	Unix domain socket for agent↔daemon IPC (Unix only).
gh-sync.json	GitHub Discussion sync state (last cursor, pushed session). Created by <code>cryo-gh init</code> .
zulip-sync.json	Zulip sync state (last message ID, pushed session). Created by <code>cryo-zulip init</code> .

10. A Complete Example Walk-Through

The following traces a single turn of a long-running coding task.

1 Operator sets up the project

```
mkdir chess-engine && cd chess-engine
cat > plan.md << 'EOF'
# Chess Engine
Implement a UCI-compatible chess engine in Rust.
Focus on alpha-beta pruning this session.
EOF
cryo init --agent "opencode run"
cryo start
```

The daemon starts; the agent wakes immediately (session 1).

2 Agent works (session 1)

```
# Inside the agent process – cryo-agent commands issued by the LLM
cryo-agent todo add "Implement move generation" --at 2026-03-07T09:00
cryo-agent note "Completed board representation. Next: move generation."
cryo-agent hibernate --summary "Board struct done. Scheduled move gen for tomorrow."
```

Daemon logs --- CRYO END ---, reads the TODO, and sleeps until 09:00.

3 Human sends a mid-sleep message

```
cryo send "Also add perft tests for debugging"
```

The daemon's inbox watcher fires and re-wakes the agent early (session 2 starts).

4 Operator checks progress

```
cryo status
# → Session: 3 | Next wake: 2026-03-08T09:00 | Agent: opencode run
cryo receive
# → Message from agent: "Move generation complete. Perft(5) = 4865609 ✓"
```

11. Provider Rotation

When multiple `[[providers]]` are defined in `cryo.toml`, the daemon rotates through them after consecutive failures. This is useful for:

- Rate-limit resilience (rotate to a different API key or model)
- Multi-model experimentation (try GPT-4 then Claude on failure)
- Budget control (cheap model first, expensive model as fallback)

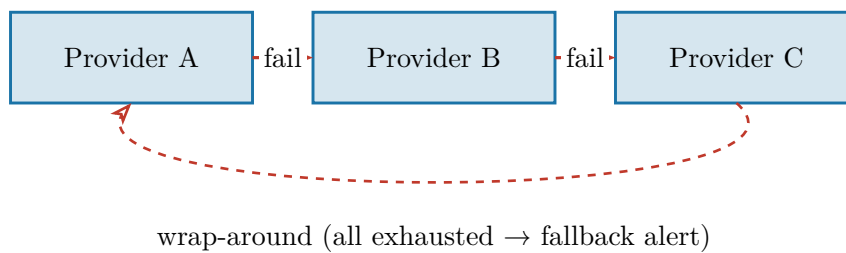


Figure 4: Provider rotation sequence on consecutive failures.

12. Tips for Developers

🔧 Development workflow

```
# run all tests
cargo test

# lint (warnings are errors in CI)
cargo clippy --all-targets -- -D warnings

# measure coverage
make coverage

# run the included mr-lazy example end-to-end
make example DIR=examples/mr-lazy
make example-cancel DIR=examples/mr-lazy
```

Key entry points for contributors:

File	Start here if you want to ...
<code>src/daemon.rs</code>	Understand or modify the session event loop, retry logic, wake detection.
<code>src/bin/cryo.rs</code>	Add or change operator-facing CLI commands.

<code>src/bin/cryo_agent.rs</code>	Add or change agent-facing IPC commands.
<code>src/platform/</code>	Port to a new OS or fix platform-specific behaviour.
<code>src/channel/zulip.rs</code>	Understand the Zulip HTTP client or add a new channel.
<code>src/agent.rs</code>	Change how agent prompts are built or how the subprocess is spawned.
<code>templates/</code>	Edit the default protocol, plan, or config injected by <code>cryo init</code> .
<code>tests/</code>	Add integration tests. CLI tests use <code>assert_cmd</code> ; HTTP tests use a minimal inline mock server.

Cryochamber is MIT-licensed. Source: <https://github.com/GiggleLiu/cryochamber>
Documentation: <https://giggleliu.github.io/cryochamber>